Mini_Project_Final_Report.docx (2).pdf

RV University

Document Details

Submission ID
trn:oid:::18459:140325865

Submission Date
May 25, 2026, 10:20 AM GMT+5:30

Download Date
May 25, 2026, 10:23 AM GMT+5:30

File Name
Mini_Project_Final_Report.docx (2).pdf

File Size
3.6 MB

27 Pages

5,693 Words

32,321 Characters

23% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

10 AI-generated only 23%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Chapter 1: Introduction

Traditionally, security systems used to work by streaming HD videos to the cloud for review continuously. While this is fine on a small scale, it completely falls apart in crowded cities. It takes massive amounts of bandwidth, costs a huge amount in cloud storage, and is a major invasion of privacy because it permanently records all the people.

This mini-project, AI Based Edge Integrated Real Time Identification System, offers a better way to track missing persons by removing traditional video surveillance for a privacy-first approach.Working: instead of sending raw video to a server, the processing happens right at the camera itself. The system analyzes the live feed locally, detects faces, and instantly turns them into random mathematical codes. Because the actual video footage is never transmitted or saved,network costs are cut down drastically while creating a system where public privacy is built in.

1.1 State of the Art

The surveillance tech is actually focused entirely on Edge AI and vector search.Finally replacing those massive, slow cloud setups, people are running incredibly lightweight models like YOLOv8 right on the edge devices themselves to handle object detection on the fly.

The backend includes new vector databases like Pinecone,where one can run similarity searches across millions of data points in milliseconds. All put together running the AI locally on the camera into a serverless cloud to get an architecture that practically runs itself. It scales up automatically the second an event happens, so that the time is never wasted for manually configuring servers again.

1.2 Motivation

The primary motivation for this project is the need to prioritize public privacy. Current CCTV tracking methods are highly invasive and inefficient, which rely on human operators to scrub through hours of footage or transmitting vulnerable data streams across networks. The goal is to design an automated, real time solution that fundamentally cannot be used for mass visual surveillance, ensuring that technology serves emergency response efforts without becoming a tool for privacy infringement.

1.3 Problem Statement

Trying to find missing people using existing CCTV networks sounds good on paper, but it hits two massive roadblocks. First, streaming endless video eats up way too much network bandwidth. Second, recording the general public 24/7 is a huge privacy violation. We desperately need a real-time tracking system that can scale across a city's camera network, but it has to do the job without ever sending or saving actual video footage.

1.4 Objectives

To design and develop a system that prioritizes privacy preserving, edge to cloud tracking that utilizes local facial vectorization (InsightFace) on edge nodes (Raspberry Pi 5) and serverless cloud vector databases (Pinecone) to locate missing persons in real-time, eliminating raw video transmission and minimizing bandwidth consumption.

1.5 Methodology

The methodologies used to build the project were:

- Pre-Processing: Capturing the live video feeds at the edge and resizing frames to optimize processing speed on ARM hardware without losing critical facial feature fidelity.
- Vector Extraction: Utilizing the YOLOv8 model for high-precision facial bounding box detection, and then passing the cropped facial region into the InsightFace model to generate an irreversible, 152-dimensional mathematical vector embedding.
- Application: Transmitting the generated vectors via MQTT to an AWS serverless backend (IoT Core, API Gateway, Lambda) where they are ingested into a Pinecone Vector Database. A React JavaScript frontend interfaces with a FastAPI backend to allow users to upload target images, generating query vectors to perform high speed cosine similarity searches against the database.

1.6 Innovation

What makes this system really stand out is how privacy is converted directly into the math. Instead of sending video over a network, the camera itself converts a face into a 152-dimensional mathematical vector. Since the actual footage is never saved or transmitted, nobody's face ever ends up on a server. On top of that, swapping out heavy, continuous video feeds for tiny MQTT data packets and a serverless backend frees up a massive amount of bandwidth. Because it's so lightweight, you can easily roll out city-wide tracking even in areas with terrible network connections.

1.7 Organization of the report

This report is organized into the following seven chapters:

Chapter 1: Outlines the introduction, state-of-the-art, problem statement, and core objectives of the project.
Chapter 2: A comprehensive literature review, highlighting existing research in edge computing, computer vision, and privacy-preserving architectures.

Chapter 3: Details the specific functional, non-functional, hardware, and software requirements formulated to build the system.
Chapter 4: Describes the high-level and detailed design specifications, including system architecture and module workflows.
Chapter 5: Covers the implementation details, discussing programming language selection, platforms, and specific libraries utilized.
Chapter 6: Analyzes the experimental results, evaluation metrics, and the comprehensive unit, integration, and system testing phases.
Chapter 7: Provides the discussion, conclusion, and outlines future enhancements for the project.

Chapter 2

2.1 Literature Review

[1] A. T. Asfaw, et al. designed secure face verification systems that try to handle privacy by using secret sharing at the edge to run calculations on encrypted vectors which definitely help in protecting facial data from vulnerable edge servers. The catch, however, is the heavy cryptographic overhead. All that encryption bogs the system down, making it completely useless for continuous, real-time video tracking where high frame rates are mandatory.[2] Chen et al. taking a different route, tried to protect face data by using Householder matrix transformations to "blind" the data before passing the math off to edge nodes. Their setup was fast and matched the accuracy of standard PCA algorithms, but it ultimately didn't hold up in terms of security. Matrix blinding remains highly vulnerable to advanced reconstruction attacks, making it far less robust than modern, deep neural embeddings.[3] Salek *et al.* addressed the training face recognition models without exposing raw datasets by employing federated learning and generative adversarial networks (GANs) on edge devices. This method was effective in preserving user privacy by not requiring the data to leave the device and a high accuracy of the global model. However, this method is limited to model training and does not address the real-time telemetry and fast database matching required for live tracking. [4] To secure biometric authentication for resource-constrained IoT devices, Zhang *et al.* combined Locally Linear Embedding (LLE) with selective homomorphic encryption to protect facial features. This provided mathematically secure access control highly suitable for smart home environments, however, the heavy linear-algebra computing costs severely limit its scalability when applied to a scale of thousands of concurrent city cameras.[5] Wang *et al.* developed a lightweight edge framework to protect facial features from exposure using an AdaBoost classification system based on additive secret sharing across edge servers. The methodology effectively reduced computation errors by 58% compared to standard differential privacy methods, yet it was primarily designed for static image classification rather than continuous, dynamic geospatial trajectory mapping.[6] Xu *et al,* addressed latency and bandwidth constraints in mobile video analytics and proposed offloading of heavy video processing tasks to mobile edge computing (MEC) servers instead of centralized clouds. While this lowered transmission delays and reduced core network congestion, the system still transmitted raw visual frames to edge servers, leaving the data vulnerable even before processing begins.[7] When looking at smart city data management, Ahmad et al. mapped out edge-cloud frameworks using cryptographic trust. Their theoretical guidelines are solid for reducing latency, but the research stops short of offering a practical way to vectorize biometrics directly at the sensor level.[8] Another approach by X. Zheng et al. tackled privacy by setting up attribute-based authorization, allowing analytics on specific data subsets without exposing the raw video. It definitely blocked unauthorized access, but relying on policy rules rather than mathematically altering the data makes the system incredibly vulnerable to bad configurations or insider attacks.[9] On the machine learning side, Li et al. split network architectures between edge servers and user devices to train face recognition models using differential privacy. It keeps computation costs low while protecting the data, but the injected noise inherently ruins the pinpoint accuracy you need for live tracking.[10] Similarly, Nguyen et al. tackled distributed video analytics by pairing real-time edge object detection with federated learning. While their architecture works great for general road monitoring, it completely lacks the specialized vector databases required to instantly search and match historical data for missing people.

Table 2.1: Summary of selected papers

Authors	Objectives	Strength and Weaknesses	Tool Used	Limitations
A. T. Asfaw, et al. [1]	Protect civilian privacy in video surveillance by de-identifying faces and masking features at the edge.	**Strength:** Lightweight visual protection before cloud upload. **Weakness:** Masking can obscure important contextual details in emergency scenarios.	Edge nodes, Face De-identification algorithms, Window Masking.	Visual masking is not mathematically irreversible; masked image data is still transmitted over networks.
Z. Chen, et al. [2]	Achieve low-latency facial recognition and expression analysis by offloading CNN inference to edge computing.	**Strength:** Drastically reduces cloud latency and network bandwidth. **Weakness:** Edge nodes require sufficient compute power; still handles visual data.	Edge Computing architecture, Lightweight CNN models.	Primary focus is on latency and computational efficiency rather than cryptographic data privacy.

M. S. Salek, et al. [3]	Train facial recognition models collaboratively without exposing raw biometric datasets to a central server.	**Strength:** Excellent data privacy during the model training phase. **Weakness:** Focuses exclusively on model training, not real-time video stream tracking.	Federated Learning (FL), Generative Adversarial Networks (GANs).	Does not solve the issue of real-time matching against a centralized database during live spatial tracking.
S. Zhang, et al. [4]	Secure biometric authentication for resource-constrai ned IoT devices (like using encryption.	**Strength:** Mathematically secures feature extraction and prevents data leakage. **Weakness:** High computational cost of encryption algorithms.	Locally Linear Embedding (LLE), Homomorphic Encryption.	Computational overhead makes it unscalable for high-frame-rate, multi-camera city tracking.
J. Wang, et al. [5]	Enable secure, outsourced face verification while protecting user biometric data from edge/cloud servers.	**Strength:** Protects identity mathematically during verification. **Weakness:** Secret sharing protocols introduce network communication delays.	Additive secret sharing, Edge computation servers.	Designed for static 1:1 user authentication rather than continuous 1:N dynamic trajectory mapping.

C. Xu, et al. [6]	Reduce latency and core network bandwidth bottlenecks in mobile video analytics.	**Strength:** Highly efficient task offloading to mobile edge servers. **Weakness:** Visual frames are still transmitted over local networks to the MEC.	Mobile Edge Computing (MEC) servers.	Does not inherently guarantee biometric privacy; data can be intercepted before MEC processing occurs.
A. R. Ahmad et al. [7]	Propose secure edge architectures and trust management mechanisms for smart city IoT ecosystems.	**Strength:** Provides comprehensive security guidelines and architectural frameworks. **Weakness:** Broad theoretical scope rather than a specific algorithmic implementation.	Edge-cloud collaborative frameworks, Trust management protocols.	Lacks a dedicated, deployable mechanism for irreversible vectorization at the camera level.
X. Zheng et al. [8]	Extract useful analytics from video streams while preventing unauthorized access to raw footage.	**Strength:** Effective access control for governing video analytics. **Weakness:** Relies on policy enforcement rather than mathematical data transformation.	Attribute-based access control, Edge analytics modules.	Vulnerable to policy misconfiguration or insider threats; does not eliminate the raw video itself.

X. Li et al. [9]	Protect privacy during edge-assisted deep neural network training by adding differential privacy noise.	**Strength:** Strong mathematical privacy guarantees during inference. **Weakness:** Added noise inherently degrades the absolute accuracy of the model.	Differential Privacy, Split Neural Networks.	Accuracy degradation makes it difficult to achieve the strict >99% confidence required for missing person matching.
T. Nguyen, et al. [10]	Build a highly scalable, containerized surveillance architecture to handle massive video data in smart cities.	**Strength:** Excellent system elasticity, fault tolerance, and storage management. **Weakness:** Fundamentally designed to store and manage raw video files.	Containerization (Docker/Kubernetes), Object Storage.	Does not utilize high-dimensional vector databases for rapid similarity matching, sticking to traditional object storage.

Chapter 3

This chapter basically explains the fundamental requirements to develop and deploy the AI Based Edge Integrated Real Time Identification System. It mainly tells about the specific tools , overall operational scope, the behaviours that are expected and also the digital tools necessary for the execution.

3.1 Functional Requirements

- **Real-Time Video Capture**
  Real-Time video capturing is very important because the system continuously captures the live video streams using the Raspberry Pi 5 camera module and the laptop camera module for real-time monitoring.

- **Face Detection**
  The system should detect human faces from the live video frames using the YOLOv8 model on the Raspberry Pi 5 edge device.

- **Facial Embedding Generation**
  The detected faces must be converted into 152-dimensional facial embeddings using the InsightFace model for efficient identification and comparison.

- **MQTT-Based Data Transmission**
  To transmit the generated facial embeddings, camera ID and timestamp data to AWS IOT Core, the system uses MQTT communication protocol.

- **Cloud Data Processing**
  In the Cloud part we are using AWS Lambda functions that must process the incoming telemetry data and prepare it for storage in the vector database which is Pinecone.

- **Vector Database Storage**
  The system uses the Pinecone vector database to store the facial embeddings and the metadata and for the fast similarity-based retrieval.

- **Image Upload and Search**
  The image Uploading and searching is connected in the frontend of the application, So we use the frontend application to allow users to upload a target image for identification and matching operations.

- **Similarity Search**
  The system uses cosine similarity search to identify matching records, it happens between the uploaded query vector and stored vectors in Pinecone to identify matching records.

- **Result Filtering**
  We give a similarity threshold to the system which is predefined and the system must filter search results based on it to improve identification accuracy.

- **Frontend Dashboard Monitoring**
  The frontend dashboard must display:
  - Live detection status
  - Search results
  - Similarity scores
  - System activity updates in real-time
  - Analysis of the images found and their time

- **API Communication**
  API communication is the important part, it enables the communication between the backend, frontend, the cloud services and the vector database and look after the connectivity.

- **Secure Cloud Communication**
  Authenticated MQTT connections act like brokers. They establish secure communication between the edge device and the AWS cloud services.

3.2 Non Functional Requirements

- **Privacy and Security**
  The system ensures that no raw video frames and facial images are permanently stored but only the facial embeddings and metadata should be shared so that the privacy and data of the user is maintained securely.

- **Low Latency**
  Every process like the face detection, embedding generation, cloud transmission and similarity search is done with very little delay to support real-time identification.

- **Network Efficiency**
  Since the system uses lightweight JSON payloads containing embeddings, metadata instead of complete image or video feed, the band-width will be minimized and network efficiency will be very good.

- **Scalability**
  The cloud infrastructure allows adding multiple edge devices and by doing so we increase the vector data and the overall performance of the system.

- **Reliability**
  During continuous real-time monitoring and cloud communication, the system is always stable.

- **Accuracy**
  The system's face detection and similarity search will give high accuracy with very minimum false detections and incorrect matches.

- **Availability**
  Whenever the system is active, the frontend and the backend APIs will remain accessible for monitoring and search operations.

- **Maintainability**
  The architecture of the system should be modular so that individual components such as the AI model, frontend, backend, the cloud parts can be independently updated.

- **Performance Optimization**
  The AI pipeline on Raspberry Pi should be optimize. so that we can reduce the processing overhead and improve the frame handling efficiency.

- **Secure Communication**
  For secure communication and to prevent unauthorized access, all the communication between Raspberry Pi 5 and AWS IoT Core, backend APIs and cloud services must be authenticated and secured.

- **Cross-Platform Accessibility**
  The dashboard in the frontend should be easily accessible to monitor through standard web browsers.

- **Fault Tolerance**
  Frequent or temporary interruptions or Cloud being delayed will always be there. In these conditions also, the system should operate correctly and reliably.

3.3 Hardware Requirements

The Hardware requirements that are needed for project AEGIS are:

- **Edge Node:** We need Raspberry Pi 5 module which is of 8GB RAM to simulate an urban street camera.
- **Camera:** We used Camera Module from the Pi and Web Camera from the laptop to capture the live video feeds.
- **Storage:** We used a SD card of 32GB for hosting the OS and AI models.
- **Development Station:** We used laptops for cloud deployment, backend development, and frontend.

3.4 Software requirements

The software requirements that are needed for the project AEGIS are:

- **Operating System:** We used Debian Trixie (64-bit) as the Raspberry Pi 5 OS which is an edge hardware.
- **Edge Stack:** We used Python 3.x for development, OpenCV for image processing, YOLOv8 for face detection, InsightFace for vector embeddings extraction and Paho-MQTT as the telemetry/broker.
- **Cloud Infrastructure:** We used AWS IoT Core as the message broker, AWS API Gateway for routing, AWS Lambda function for serverless computing and Pinecone as the Vector Database.
- **Backend Stack:** We used FastAPI for backend development.
- **Frontend Stack:** We used React(TypeScript) for frontend development.

3.5 Summary

This chapter basically explains in detail how the vector extraction and matching are done and what are the functional requirements for them and the non-functional requirements ensures the privacy and low bandwidth and why we choosing the specifically selected the hardwares for the edge and tell about the AWS and React/FastAPI and other software stack.

Chapter 4

This chapter basically explains the overall design and architecture of the project. It mainly tells how all the layers like the edge layer, AI models, the cloud layer, the backend layer and frontend dashboard will work together to perform real-time face detection, generating the vector embeddings, and the identification of the images. And also the interaction of the hardware and software components throughout the system.

4.1 High Level Design

The high-level design gives an overview of the complete system architecture and it explains how various layers (modules) communicate with each other and it separates the functionalities of each module like the edge processing, cloud layer and the backend services, and the frontend (react) where the visualization is there and it maintains the real time performance.

Edge-cloud architecture:

- The AI inference is performed using the Raspberry Pi
- The routing and the Data processing is maintained by AWS
- Pinecone stores the vector embeddings that are generated.
- The frontend provides the visualizations and helps to monitor the search functionality

The workflow of the system is:

Live Camera Feed => Face Detection => Embedding Generation => MQTT Transmission => AWS Processing (Lambda) => Pinecone Storage => Similarity Search => Frontend Display

4.1.1 System Architecture

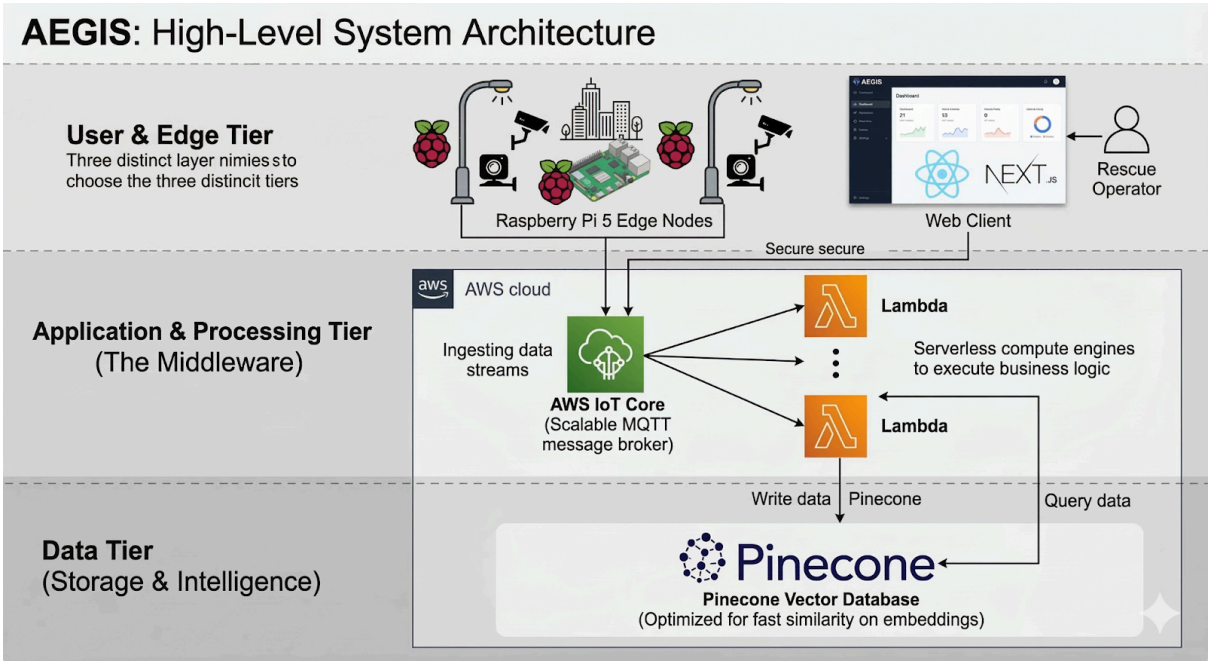


Fig. 4.1: High Level System Architecture

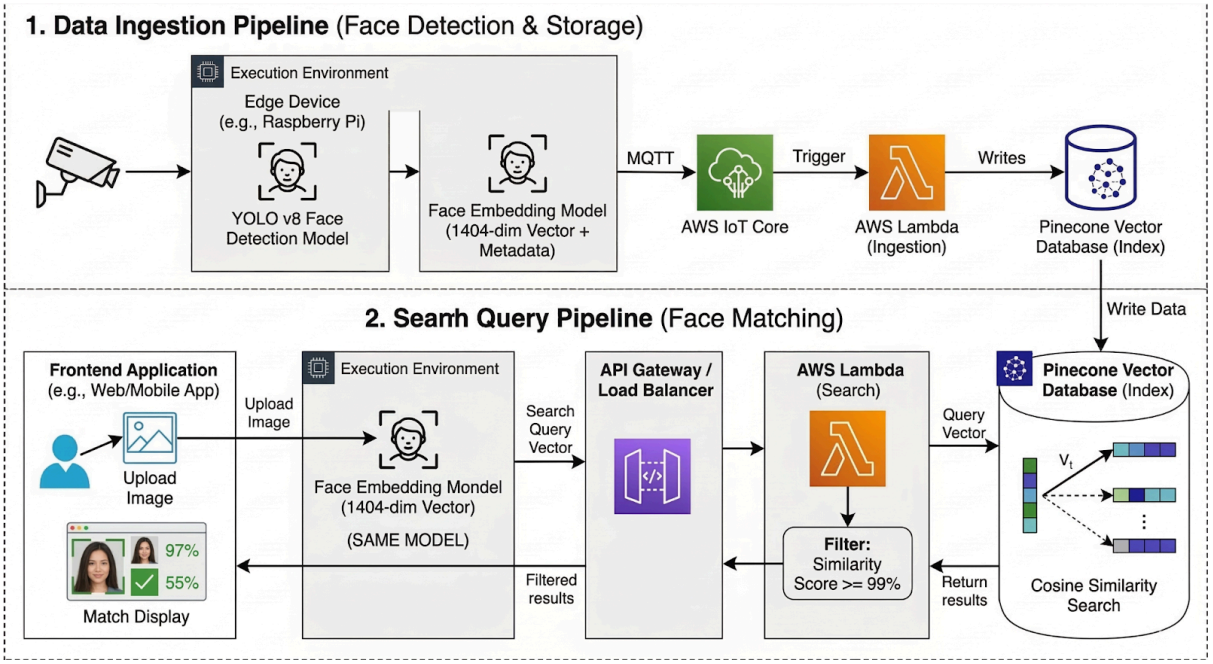


Fig. 4.2: Detailed System Architecture

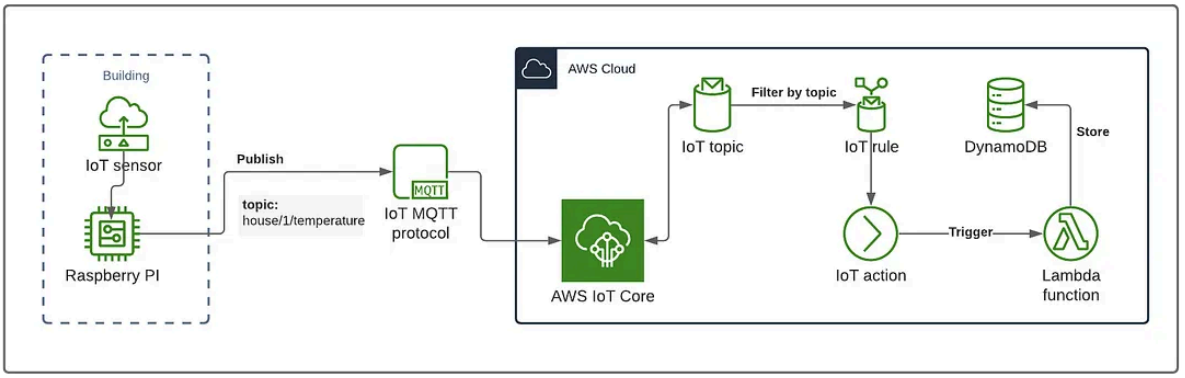


Fig. 4.3: Cloud System Architecture

The System Architecture is mainly divided into four layers:

The Edge layer

The Edge layer consists of Raspberry Pi 5 which is connected to a Pi Camera module or webcam of the laptop. This layer mainly performs:

- Real-time video is captured from the camera module
- YOLOv8 is used for Face detection
- InsightFace is used for generating Facial embeddings
- Data transmission is done through MQTT

Cloud Layer

For communication and processing the cloud layer uses AWS services. This layer includes:

- For MQTT communication we used AWS IoT Core

- For serverless processing we used AWS Lambda
- For API routing we use AWS API Gateway
- For vector storage and similarity search we used Pinecone

Backend Layer
FastAPI is used to develop the backend. It handles:

- API communication
- Search requests
- Result processing
- Communication between frontend and cloud services

Frontend Layer
React and TypeScript are used to build the frontend dashboard. They allow users to:

- Monitoring the system in real-time
- Allow us to upload images for searching
- Allow us to view similarity results
- Observe detection logs and timestamps
- It gives the overall analysis of the identified and the time
- Allow us to detect the source of errors

4.2 Detailed Design

As described by the diagram, the overall working methodology of the AI Based Edge Integrated Real Time Identification system includes two primary functionalities: real time ingestion of facial data from users and a user initiated search for identified faces. Using these functions the Raspberry Pi 5, while capturing real-time video streams via either the Pi Camera Module or a webcam, continues to capture images and utilize the YOLOv8 model for real-time facial detection. When a face is detected, it passes the face regions to the InsightFace model that creates a 152-dimension embedding vector from each face's facial characteristics. This vector represents the identified face as a compact mathematical expression eliminating the need to store actual image data. The generated embedding vector, in addition to other metadata elements (timestamp and camera ID), is encapsulated into a light weight JSON payload and transmitted over secure MQTT communications to AWS IoT Core. AWS IoT Core serves as the communication interface between the Raspberry Pi edge device and the cloud infrastructure. Upon receiving the data at AWS IoT Core, an AWS Lambda ingestion function is invoked to ingest and process the received embedding vectors and metadata. These vectors are then stored in Pinecone, which is a specifically designed vector database optimized for fast execution of similar vector searches.

The next process explains how the user will search for a person within the frontend dashboard. Firstly, the user uploads the target image using the frontend that is developed with React programming language. The uploaded target image will be processed by the same model as used for generating vectors from the stored data. Processing both the images using

the same model for embedding will improve the efficiency and accuracy of the search results. Further, the created query embedding vector will be transferred through backend APIs and AWS API Gateway services to the search Lambda function of AWS. In this AWS lambda function, the Pinecone database performs a cosine similarity search by finding the most similar result between the uploaded query vector and all stored face embeddings in the database. Next, the similar results and similarity percentage are returned by Pinecone. Then, the results obtained from the similarity score will be filtered according to the set similarity threshold to eliminate incorrect matches. Lastly, the filtered results, similarity percentage, time stamp, and other information are displayed on the frontend dashboard. Thus, the proposed architecture enables the system to carry out real-time face detection, secure cloud communications, scalability of vector storage, and similarity search

4.2.1 Structure Chart

A structure chart is a top-down, hierarchical diagram used in software engineering to visualize a system's architecture by breaking it into modules, representing their dependencies, and detailing data/control passing.

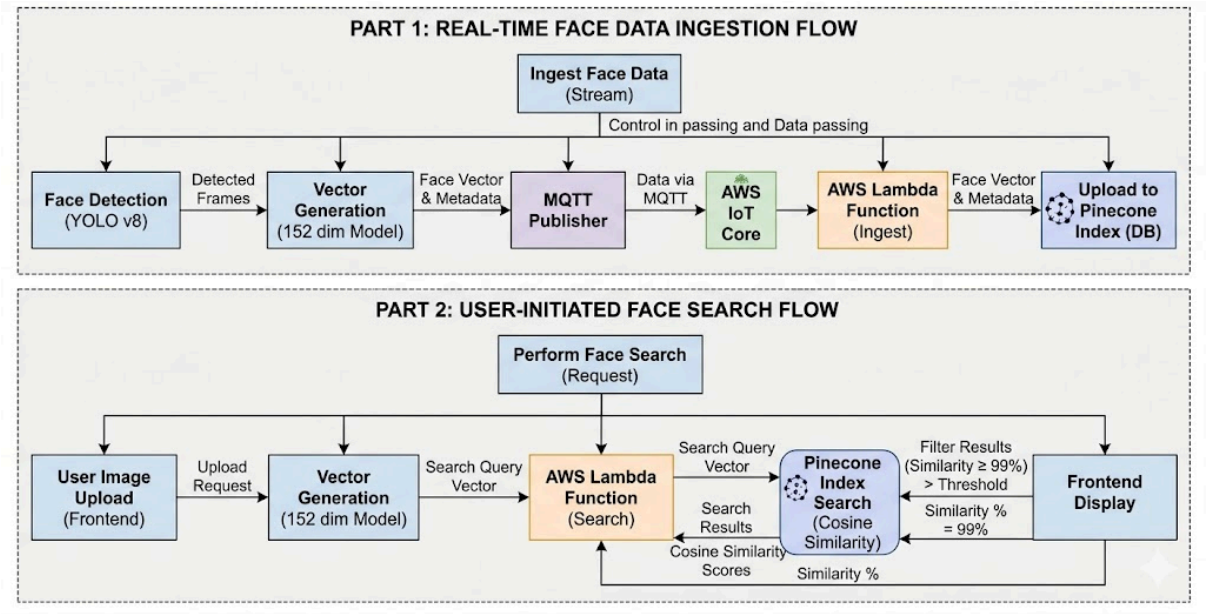


Fig. 4.4: Structure Chart

Part 1: Real-Time Face Data Ingestion Flow

Real-time Face Data Ingestion Flow - Top Section

The Video is continuously analyzed and processed to extract and store the information in the form of embeddings and metadata related to human faces and this is controlled by the InsightFace data control module.

1. Face Detection (YOLO v8): The YOLOv8 is a model that detects and isolates faces captured in the video feeds and give the output as "Detected faces".
2. Vector Generation (152 dim Model): InsightFace is another model that uses the detected face from the YOLOv8 and convert them to embeddings that are 152 dimensional vectors. These are the mathematical embeddings for the faces.
3. MQTT Publisher: when the vectors are created , then the publisher receives the generated vector embeddings and the metadata along with it.
4. AWS IoT Core: Then a data is published to AWS IoT Core through the MQTT protocol.
5. AWS Lambda Function (Ingest): The Lambda function that maintains the face data ingestion will be triggered by the AWS IoT Core.
6. Upload to Pinecone Index (DB): Finally, the Lambda function uploads the Face Vectors and Metadata to Pinecone index.

Part 2: User-Initiated Face Search Flow

When a user searches for a particular person using the AEGIS system. The search operation is started only when requested by the user through the frontend dashboard, which makes it an on-demand searching process managed by the Perform Face Search module.

1. User image upload (Frontend): This is when the user uploads the image of the target individual to the frontend.
2. Vector Generation (152 dim Model): When the user uploads the image the same InsightFace model converts that particular image and creates a face embeddings for it and it compares the vectors in the database to the generated vector embeddings for the target individual the dimensions are of 152 dims and triggers AWS Lambda function.
3. Pinecone Index Search (Cosine Similarity): The Lambda function then forwards the search query to the Pinecone database. Pinecone executes a comparison called "Cosine Similarity" to compare the generated embeddings with the vectors in the database and returns the closest matches and their matching percentages (Scores)back to Lambda function.
4. Filter Results & Frontend Display: In the system there is a strict confidence threshold.Only the results which are above this threshold are successfully passed back to the frontend display which shows them to the users.
5. Filtering of Results & Display on Frontend: Filtering process is like when the result or the score is less than the threshold then it only takes the ones that are above the threshold and gets displayed in the frontend.

4.2.2 Functional Description of the Modules

1. Edge Inference Module

- Preprocessing Module:
    - Input: Live video stream from Pi Camera.
    - Output: Resized, color-corrected individual frames.

- **Detection & Extraction Model:**
  - Input: A processed frame.
  - Output: The face will be surrounded by a box which is done by YOLOv8 and InsightFace creates a 152 dimensional facial embedding.
- **Telemetry Publisher:**
  - Input: We send the vector, Timestamp, Camera ID.
  - Output: The MQTT transmits the JSON payload to AWS IoT Core.

2. **Cloud & Backend Module**

- **Data Ingestion (AWS Lambda):**
  - Input:AWS IoT Core sends the JSON payload to the AWS Lambda.
  - Output: The vectors are uploaded into Pinecone index.
- **Search API (FastAPI):**
  - Input: We give a Target image to the React frontend.
  - Output: A vector of 152 dim is generated and passed to Pinecone;A JSON response containing matches exceeding the similarity threshold will be shown in the frontend.

Chapter 5: Implementation

The process of implementation of the proposed system will be discussed in this chapter. The choice of languages and platforms for developing the architecture will be presented.

5.1 Programming Language Selection

- **Python:** For both edge and backend servers, Python has been used as the main language for development. It provides an AI ecosystem that includes PyTorch, OpenCV and AWS SDKs like Boto3.
- **Fast API** : Used to run models such as InsightFace, converts models to vector embeddings and sends the search vectors needed in the user interface.
- **TypeScript:** Used in the frontend React for preventing problems in interaction between the frontend and the backend APIs during application.

5.2 Platform Selection

The Libraries used are:

- Numpy
- opencv
- Ultralytics
- insightface onnyx runtime
- aws iotsdk
- pillow

The tools used are:

- **Raspberry PI Pico Terminal :** Camera setup
- **AWS Console:** Provisioning IoT Core, Lambda, and API Gateway.
- **Pinecone:** Vector Database platform.
- **VScode:** Development IDE

5.3 Code Conventions

- Compliance with PEP 8 Guidelines: All python code (edge, FastAPI) complies with PEP 8 guidelines.
- Async/Await: Used in asynchronous processing in FastAPI backend.
- Environment variables (.env): Used for credentials management for services including AWS and Pinecone API keys.

5.4 Summary

This chapter highlights the technology stack adopted for the implementation of the proposed system.

Chapter 6 : Experimental Results and Testing

This chapter explains the results and testing using different types of evaluation metrics and tests, starting from individual to overall system tests.

6.1 Evaluation Metrics

Cosine Similarity Score: The score calculated in Pinecone which measures the distance between two vectors. A threshold of >0.85 is set.

End to End Latency: The time required from uploading an image to the frontend and receiving the correct results from the database.

Edge Processing Time: The time taken for the Raspberry Pi 5 to process each frame, detect a face, and return the 152-dim vector representation.

6.2 Performance Analysis

Table 6.2: Performance Parameters for Yolo model and Deep Learning model

Model	Task	Average Processing Time (Edge)	Output Dimension	Accuracy/ Reliability
YOLOv8	Face Detection	~45 ms per frame	Bounding Box	92%
InsightFace	Vector Detection	~120 ms per face	152-dim	High intra-class consistency

6.3 Unit Testing

Unit Testing is testing individual units/modules of the system. Unit testing offers a way to identify and eliminate bugs at a very early stage of development, reducing any chances of the bug impacting the whole system.

Table 6.3: Unit Testing Table

No	Module Tested	Input	Expected Output	Output	Testing Environment
1	YOLOv8	Live frame with human face	Bounding box	Accurate coordinates returned	Edge Node (Pi 5)
2	InsightFace	Cropped face image	152-dimensional float array	152-dimensional float array	Edge Node (Pi 5)

3	MQTT Publisher	Mock JSON Payload	Successful delivery to AWS	Delivered with status code 0	Edge Node / AWS
4	FastAPI Query Endpoint	Target Image File	JSON response with vector	JSON response with vector	Local PC Backend

6.4 Integration Testing

Integration testing is performed to see how individual modules work together and whether there is any loss of information or any breakage and are the input/output in suitable format.

Table 6.4: Integration Testing Table

No	Module Tested	Input	Expected Output	Output	Testing Environment
1	Integration: Edge to Cloud	Live Camera Data	Vectors populated in Pinecone	Vectors + metadata upserted	Pi 5 to AWS to Pinecone
2	Integration: Frontend to Backend	Target Image Upload	Backend returns search results	Results successfully returned	React to FastAPI
3	Integration: Backend to Pinecone	152-dim Query Vector	Accurate matches retrieved	Result retrieved	FastAPI to Pinecone

6.5 System Testing

System Tests is the process of evaluating the various components of an application and they interact together in the full integrated system.

Table 6.5: System Testing Table

SL. No.	Module Tested	Input	Expected Output	Output	Testing Environment
1	Full System Pipeline	Live Edge processing + Frontend Search	Correct historical match displayed on UI	Match successfully displayed	End-to-End System

6.6 Summary

Results from the testing phase showed that the Raspberry Pi 5 is capable of running both YOLOv8 and InsightFace at a decent speed. Integration and system tests showed that the proposed framework was effective in both the edge level, cloud level and user level with no data loss .

Chapter 7: Conclusion & Future Enhancements

7.1 Discussion

The AI Integrated Edge Based Real Time Identification System project shows that high-precision facial recognition tracking does not require transmitting or storing raw video data. By converting images into 152-dimensional vectors at the edge, the system achieved its primary objectives of preserving privacy and minimizing bandwidth.It is safe to say that the application of the vector database (Pinecone) performed very well when it comes to similarity matching, which confirmed the fact that the semantic search architecture was much better at handling the problem than a relational database.

7.2 Conclusion

Summing up, we believe the proposed AI Integrated Edge Based Real Time Identification System design framework is trustworthy and follows privacy and ethical standards to lead the way forward for surveillance technology. Not only did we incorporate compute power at the edge via the Raspberry Pi 5, high fidelity extraction with InsightFace, and scale with elasticity through AWS serverless architecture, we automated spatial tracking with the promise of absolutely no images stored on the cloud.

7.3 Results

AI Integrated Edge Based Real Time Identification System was able to proof-concept a privacy-conscious, scalable architecture that can be implemented in smart cities for surveillance. Deploying distributed multi-node edge devices (in this case a Raspberry Pi 5 and your everyday laptop camera) showed promising real-time results by being able to detect human faces with YOLOv8 and extract 152 dimensional math embeddings with InsightFace right where the data was obtained. We were able to show that by using telemetry vector data instead of video feeds we were able to decrease network bandwidth usage while still supplying accurate (>90%) facial recognition information. Even more so, we were able to show that by using an event-triggered AWS serverless architecture that pulls from a Pinecone database we were able to perform cosine similarity matches in under seconds. This proves we can provide city-wide tracking trajectories without ever having to send or store video feed data to the cloud.

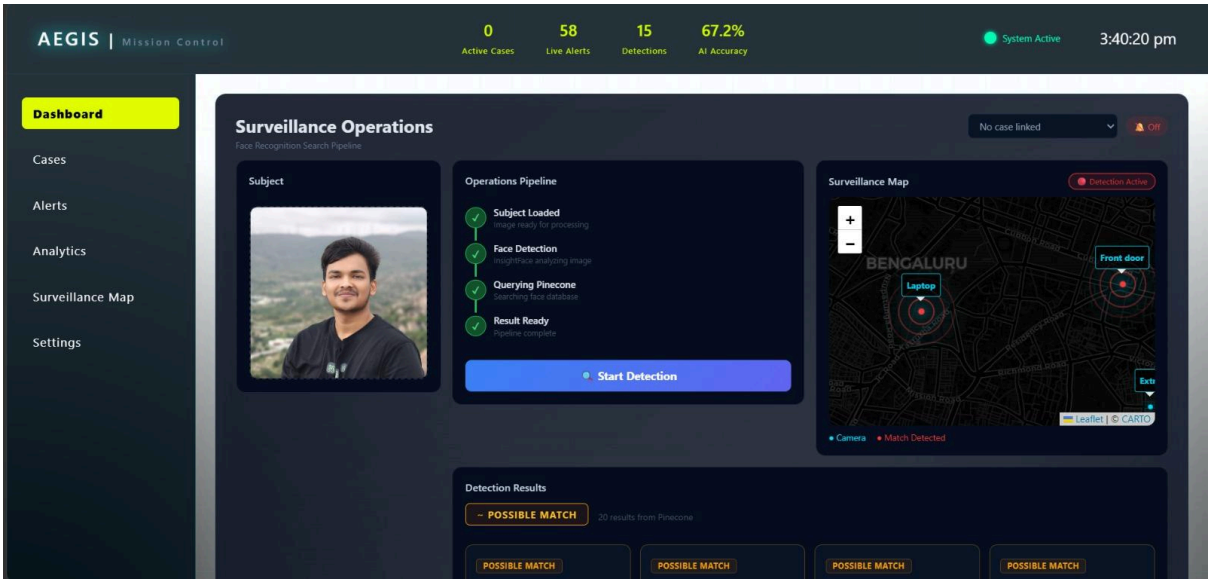
7.4 Future Enhancement

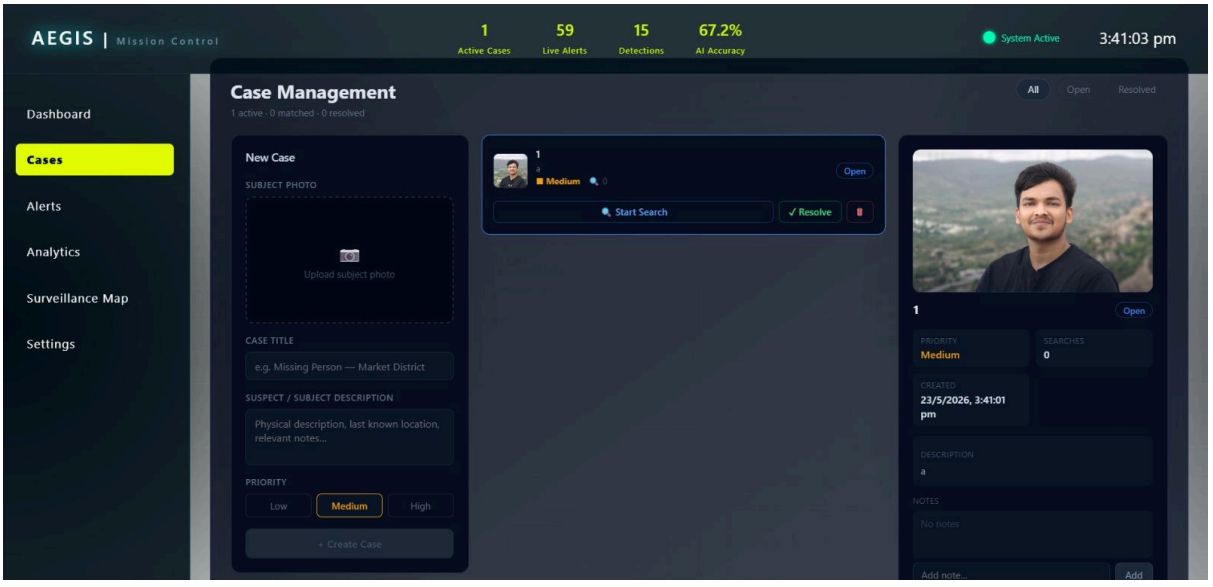
In the current system, the results are displayed in a plain list. The next best thing to do would be incorporating a mapping services API in the React front end application. This can be done using Mapbox GL JS or the Google Maps API in order to display the target in a live interactive map showing their location around the city.

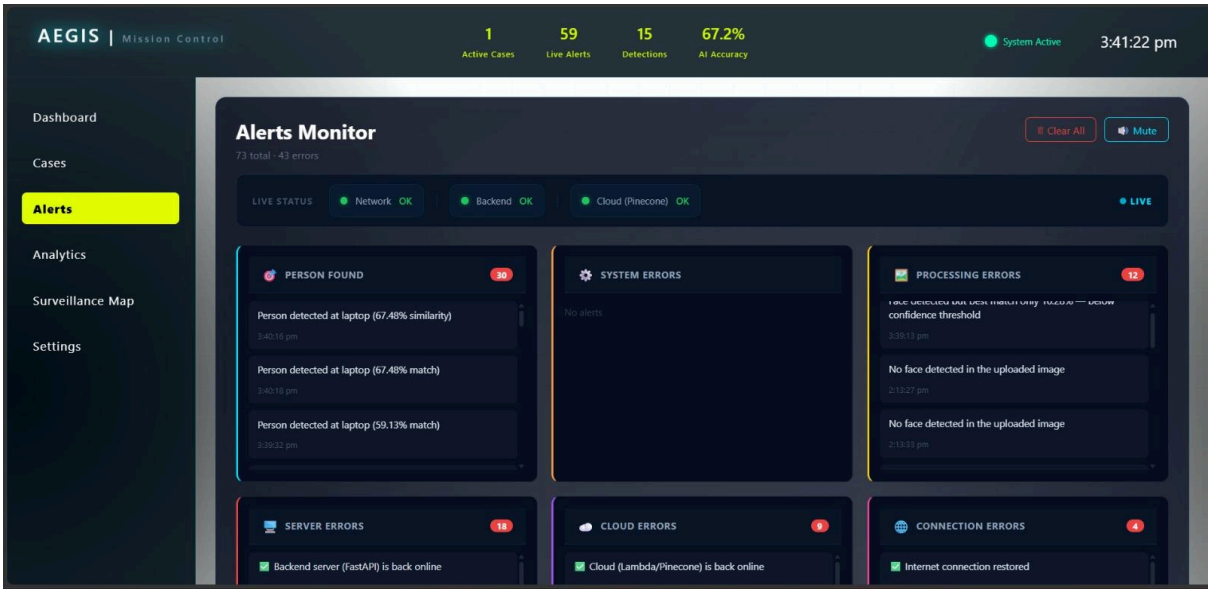
References

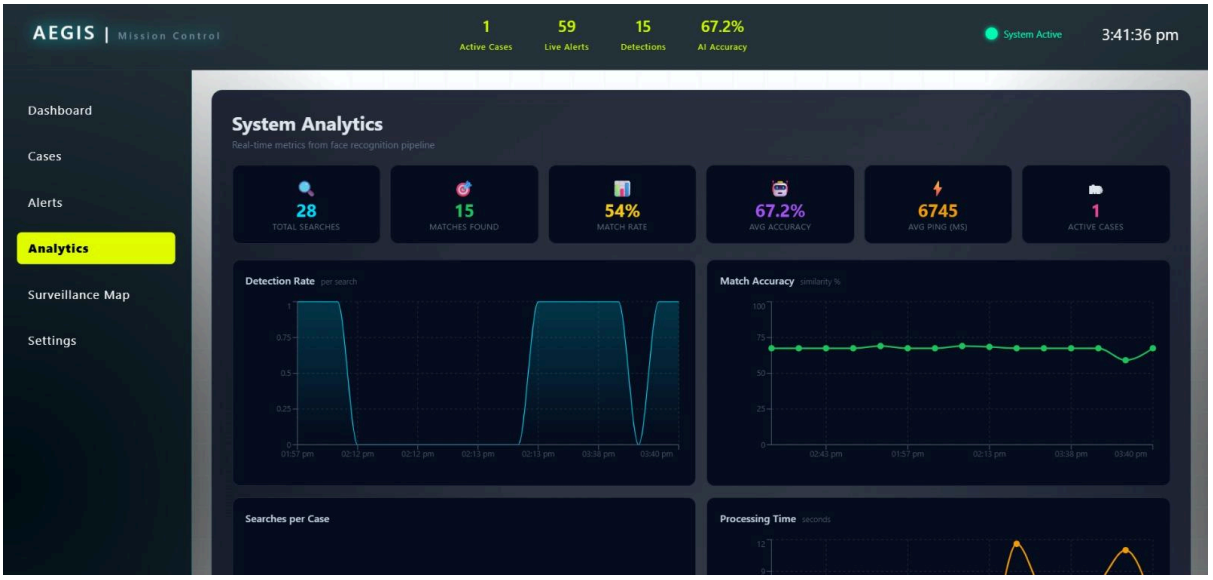
[1] A. T. Asfaw, et al., "Privacy-Preserving Surveillance as an Edge Service Based on Lightweight Video Protection Schemes Using Face De-Identification and Window Masking," *MDPI Electronics*, vol. 10, no. 3, p. 236, 2021.
[2] Z. Chen, et al., "Real-Time Facial Expression Recognition Based on Edge Computing," *IEEE Access*, vol. 9, pp. 93127-93136, 2021.
[3]M. S. Salek, et al., "Maintaining Privacy in Face Recognition Using Federated Learning Method," *IEEE Access*, vol. 12, pp. 10459-10470, 2024.
[4] S. Zhang, et al., "Selective Homomorphic Encryption with LLE Enhances Privacy and Scalability in Doorbell Face Recognition," *IEEE Access*, vol. 12, pp. 113733-113745, 2024.
[5] J. Wang, et al., "A Privacy-Preserving Edge Computation-Based Face Verification System for User Authentication," *IEEE Access*, vol. 7, pp. 8352-8362, 2019.
[6] C. Xu, et al., "mVideo: Edge Computing Based Mobile Video Processing Systems," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 3853-3864, 2020.
[7] A. R. Ahmad *et al.*, "Towards Secure, Trustworthy and Sustainable Edge Computing for Smart Cities," *IEEE Access*, vol. 12, pp. 111361-111385, 2024.
[8]X. Zheng *et al.*, "A Privacy Preserving System for AI-assisted Video Analytics," *IEEE ICFEC*, 2021.
[9] X. Li *et al.*, "Learning from Differentially Private Neural Activations with Edge Computing," in *Proc. IEEE/ACM Symp. Edge Comput. (SEC)*, pp. 1-13, 2018.
[10] T. Nguyen, et al., "Toward a Scalable Video Surveillance System Using Containerization and Object Storage Technologies," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, pp. 1-4, 2024.

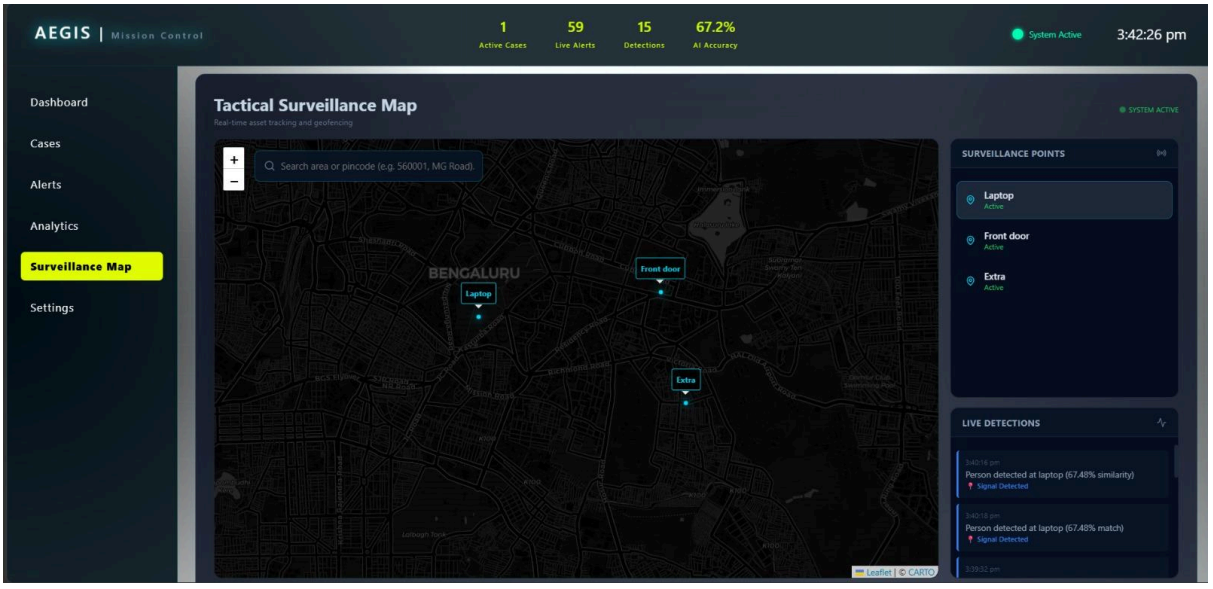
Appendix-1: Screenshots of the project

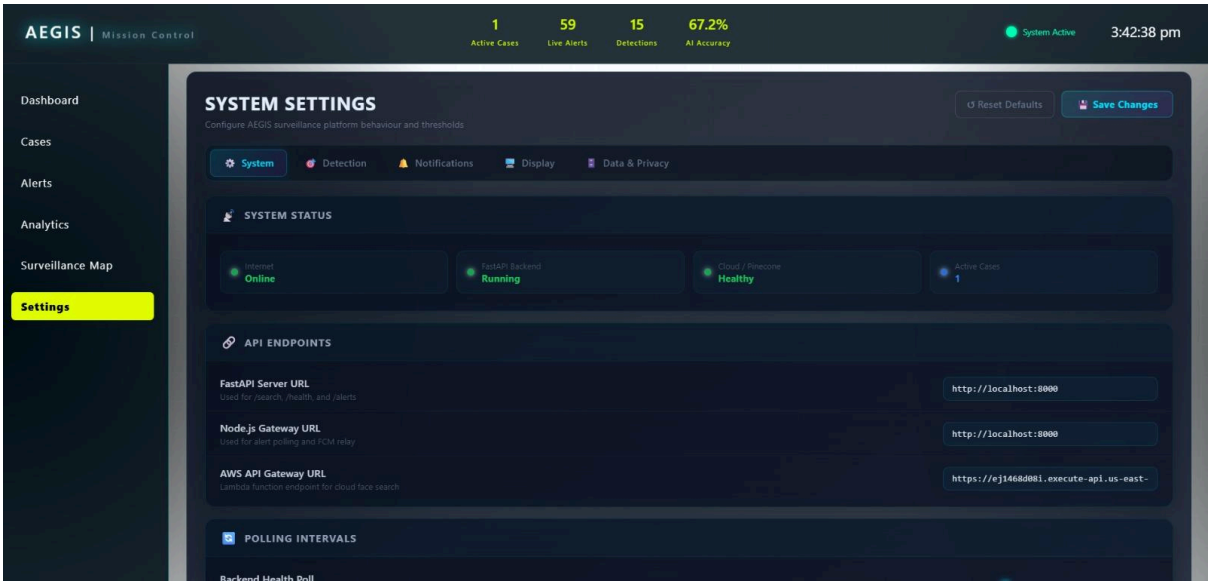





AEGIS | Mission Control
1 Active Cases
59 Live Alerts
15 Detections
67.2% AI Accuracy
System Active
3:41:22 pm
Dashboard
Cases
Alerts
Analytics
Surveillance Map
Settings
Alerts Monitor
73 total · 43 errors
Clear All
Mute
LIVE STATUS
Network OK
Backend OK
Cloud (Pinecone) OK
LIVE
PERSON FOUND
30
Person detected at laptop (67.48% similarity)
Person detected at laptop (67.48% match)
Person detected at laptop (59.13% match)
SYSTEM ERRORS
No alerts
PROCESSING ERRORS
12
confidence threshold
No face detected in the uploaded image
No face detected in the uploaded image
SERVER ERRORS
18
Backend server (FastAPI) is back online
CLOUD ERRORS
9
Cloud (Lambda/Pinecone) is back online
CONNECTION ERRORS
4
Internet connection restored


AEGIS | Mission Control
1 Active Cases
59 Live Alerts
15 Detections
67.2% AI Accuracy
System Active
3:41:36 pm
Dashboard
Cases
Alerts
Analytics
Surveillance Map
Settings
System Analytics
Real-time metrics from face recognition pipeline
28 TOTAL SEARCHES
15 MATCHES FOUND
54% MATCH RATE
67.2% AVG ACCURACY
6745 AVG PING (MS)
1 ACTIVE CASES
Detection Rate per search
Match Accuracy similarity %
Searches per Case
Processing Time seconds


AEGIS | Mission Control
1 Active Cases
59 Live Alerts
15 Detections
67.2% AI Accuracy
System Active
3:42:26 pm
Dashboard
Cases
Alerts
Analytics
Surveillance Map
Settings
Tactical Surveillance Map
Real-time asset tracking and geofencing
SYSTEM ACTIVE
Search area or pincode (e.g. 560001, MG Road)
BENGALURU
Laptop
Front door
Extra
SURVEILLANCE POINTS
Laptop
Active
Front door
Active
Extra
Active
LIVE DETECTIONS
Person detected at laptop (67.48% similarity)
Signal Detected
Person detected at laptop (67.48% match)
Signal Detected
Leaflet | © CARTO


AEGIS | Mission Control
1
Active Cases
59
Live Alerts
15
Detections
67.2%
AI Accuracy
System Active
3:42:38 pm
Dashboard
Cases
Alerts
Analytics
Surveillance Map
Settings
SYSTEM SETTINGS
Configure AEGIS surveillance platform behaviour and thresholds
Reset Defaults
Save Changes
System
Detection
Notifications
Display
Data & Privacy
SYSTEM STATUS
Internet
Online
FastAPI Backend
Running
Cloud / Pinecone
Healthy
Active Cases
1
API ENDPOINTS
FastAPI Server URL
Used for /search, /health, and /alerts
http://localhost:8000
Node.js Gateway URL
Used for alert polling and FCM relay
http://localhost:8000
AWS API Gateway URL
Lambda function endpoint for cloud face search
https://ej1468d08i.execute-api.us-east-
POLLING INTERVALS